

Pion qTMD and PDF Measurement

Comprehensive Physics Note

Pyquda_Measurement Collaboration

July 18, 2026

Contents

1	Physics Overview	2
1.1	Transverse Momentum Dependent Distributions	2
1.2	Equal-Time Correlators on the Lattice	3
2	Quasi-PDF and Quasi-TMD: LaMET Framework	3
3	Two-Point Function	3
3.1	Physical Definition	3
3.2	Source Gamma Convention	4
3.3	Einsum Contraction Pattern	4
3.4	Boosted Smearing	5
4	Three-Point qTMD Function (Pre-Sequential-Trick)	5
4.1	Physical Correlator	5
4.2	Why the Sequential Trick is Needed	5
5	Sequential Source	6
5.1	Meson Sequential Source Construction	6
5.2	Sink-Summed Three-Point Contraction	6
6	CG qTMD Operator	7
6.1	Coordinate-Gauge Displacement	7
6.2	Transverse Direction Scan	7
6.3	Wilson Line Index Convention	7
7	PDF Operators ($b_T = 0$)	7
7.1	CG PDF (Coordinate Gauge)	7
7.2	GI PDF (Gauge Invariant)	8
8	Wilson Line Index Lists	8
8.1	TMD Wilson Line Indices	8
8.2	Reordering for Minimal Adjacent Differences	8
8.3	PDF Wilson Line Indices	9
9	Code Implementation Details	9
9.1	<code>__init__(parameters)</code>	9
9.2	<code>contract_2pt_pion</code>	9
9.3	<code>contract_qTMD_CG</code>	9
9.4	<code>contract_PDF</code>	10

9.5	Shared per-shift Gamma scan	10
9.6	shift_qtmd_cg	10
9.7	shift_propagator_pdf_gi	11
9.8	meson_backward_line	11
10	Output Shape Convention and HDF5 Layout	11
10.1	Raw Contraction Shape	11
10.2	Explicit Transpose for HDF5 Write	11
10.3	Source Time Rolling	11
10.4	HDF5 Directory Layout	12
10.5	Lightweight Source-Level Resume	12
11	Boost Smearing Parameters	12
12	Comparison with Proton TMD	13
12.1	Flavor Dimension	13
12.2	Spin Polarization	13
12.3	Contraction Structure	13
12.4	Sequential Source	13
12.5	Summary Table	14
13	Summary of Key Formulae	14
A	Gamma Matrix Reference	14
B	Wilson Line Index Convention	15

1 Physics Overview

1.1 Transverse Momentum Dependent Distributions

Transverse momentum dependent (TMD) parton distribution functions and parton distribution functions (PDFs) encode the three-dimensional momentum structure of hadrons. The TMD correlator for a quark in a hadron is defined on the light cone,

$$\Phi_q(x, \mathbf{k}_\perp; P, S) = \int \frac{d\xi^- d^2\xi_\perp}{(2\pi)^3} e^{ik \cdot \xi} \langle P, S | \bar{\psi}(0) \mathcal{W}(0, \xi; \text{path}) \psi(\xi) | P, S \rangle \Big|_{\xi^+=0}, \quad (1)$$

where $\mathcal{W}(0, \xi; \text{path})$ is the staple-shaped Wilson line connecting the quark fields, P is the hadron momentum, and S its spin.

On the lattice, we work at equal time and replace the light-cone separation with a purely spatial displacement. The quasi-TMD correlator in the rest frame of the temporal direction is

$$\tilde{\Phi}_\Gamma(b_\perp, b_z, p^z; \tau) = \langle \pi(p^z) | \bar{\psi}(0) \Gamma \mathcal{W}_{\text{CG}}(0; b_\perp, b_z) \psi(\tau; b_\perp, b_z) | \pi(p^z) \rangle. \quad (2)$$

Here $\mathbf{b}_\perp = b_\perp \mathbf{e}_\perp$ is the transverse displacement (in the x or y direction), $b_z \mathbf{e}_z$ is the longitudinal displacement along the boost direction, τ is the operator insertion time, and $\mathcal{W}_{\text{CG}}$ denotes the coordinate-gauge Wilson line (or a true gauge-covariant staple for the operator with explicit gauge links). Γ is a Dirac matrix selecting the desired spin structure.

1.2 Equal-Time Correlators on the Lattice

On the Euclidean lattice the time coordinate is Wick-rotated: $t = i\tau$. The equal-time operator insertion means the quark and antiquark fields in the nonlocal bilinear are evaluated at the same time slice τ (or equivalently the same Euclidean time t). The spatial displacement $\mathbf{b}$ is realized as a lattice shift operator acting on the quark propagator.

For the pion, which is a spin-zero meson, only specific Γ projections survive and the TMDs are spin-averaged. The pion qTMD distribution probes the transverse and longitudinal momentum correlations of the valence quark and antiquark within the pion.

2 Quasi-PDF and Quasi-TMD: LaMET Framework

Physical PDFs and TMDs are defined on the light cone, while lattice QCD computes correlation functions at equal time. Large momentum effective theory (LaMET) [1, 2] bridges the two: one computes a *quasi-distribution* on the lattice at finite hadron momentum p^z , then matches it to the physical distribution through a factorization formula

$$\tilde{q}(x, b_\perp, p^z; \mu) = \int \frac{dy}{|y|} C\left(\frac{x}{y}, \frac{\mu}{p^z}\right) q(y, \mu) + \mathcal{O}\left(\frac{\Lambda_{\text{QCD}}^2}{(p^z)^2}, \frac{\Lambda_{\text{QCD}}^2}{b_\perp^2}\right). \quad (3)$$

The pion boost momentum $\mathbf{p}_f = (0, 0, p^z)$ is a crucial parameter. Three regimes must be controlled:

- (i) The boost momentum must be large enough that power corrections $\Lambda_{\text{QCD}}^2/(p^z)^2$ are under control.
- (ii) The spatial displacement b_z must be small compared to the correlation length of the gauge links (to avoid large noise from Wilson line self-energy).
- (iii) The transverse displacement $b_\perp$ must be large enough to resolve non-perturbative transverse structure but small enough to avoid discretization effects.

In this workflow, the quasi-TMD is computed directly from the lattice three-point function with a nonlocal displacement operator inserted between the source and the fixed sink. The equal-time correlator is

$$\tilde{C}_3^\Gamma(q, b_\perp, b_z; \tau; p_f, t_{\text{sep}}) = \sum_{\mathbf{x}} \sum_{\mathbf{y}} e^{-i\mathbf{q} \cdot (\mathbf{x} - \mathbf{x}_0)} e^{-i\mathbf{p}_f \cdot (\mathbf{y} - \mathbf{x}_0)} \langle \pi(\mathbf{y}, t_{\text{sep}}) | \bar{\psi}(\mathbf{x}, \tau) \Gamma \mathcal{O}_{b_\perp, b_z} \psi(\mathbf{x}, \tau) | \pi(\mathbf{x}_0, 0) \rangle. \quad (4)$$

The momentum $\mathbf{p}_f$ is the fixed sink momentum that determines the pion boost; $\mathbf{q}$ is the Fourier momentum at the operator insertion; and the initial pion momentum $\mathbf{p}_i$ is related by momentum conservation at the insertion: $\mathbf{p}_f = \mathbf{p}_i + \mathbf{q}$ when the operator carries zero momentum (purely spatial displacement). In practice p_f is set to the desired boost and q scans all momentum modes of the lattice for the non-forward matrix element.

3 Two-Point Function

3.1 Physical Definition

The connected pion two-point function for source gamma Γ_{src} and sink gamma Γ_{sink} is

$$C_2^{\Gamma_{\text{sink}}}(p, t) = \sum_{\mathbf{x}} e^{-i\mathbf{p} \cdot (\mathbf{x} - \mathbf{x}_0)} \text{Tr}_{\text{spin, color}} \left[S_{\text{anti}}(\mathbf{x}, t; \mathbf{x}_0, 0) \Gamma_{\text{sink}} S_q(\mathbf{x}, t; \mathbf{x}_0, 0) \Gamma_{\text{src}} \right], \quad (5)$$

where $S_q(x, y)$ is the quark propagator from source y to sink x , and S_{anti} is the antiquark line constructed via γ_5 -hermiticity:

$$S_{\text{anti}}(x, y) = \gamma_5 S_q(x, y)^\dagger \gamma_5. \quad (6)$$

3.2 Source Gamma Convention

The default source gamma is the explicit canonical label 5, which sets $\Gamma_{\text{src}} = \gamma_5$ for all sink channels and corresponds to the standard pseudoscalar pion interpolator $\bar{\psi}\gamma_5\psi$. The shared source-Gamma utility supports:

- Any of the 16 canonical Gamma labels: the selected source matrix is fixed while all 16 sink channels are scanned.
- **dagger_of_sink**: $\Gamma_{\text{src}} = \gamma_5\Gamma_{\text{sink}}^\dagger\gamma_5$.

The former aliases **fixed_g5** and **same_as_sink** are not accepted. Explicit 5 replaces **fixed_g5** without changing the numerical contraction. The relational **dagger_of_sink** mode is used by qDA local C2; it produces 16 paired source/sink channels rather than a fixed-source sink scan. For an explicit pseudoscalar source, output provenance uses **source_gamma_mode=fixed**, **source_gamma_label=5**, and the filename channel tag **.src5**.

All 16 sink gamma matrices are scanned and stored, ordered as

$$\gamma_5, \gamma_T, \gamma_T\gamma_5, \gamma_X, \gamma_X\gamma_5, \gamma_Y, \gamma_Y\gamma_5, \gamma_Z, \gamma_Z\gamma_5, I, \Sigma_{XT}, \Sigma_{XY}, \Sigma_{XZ}, \Sigma_{YT}, \Sigma_{YZ}, \Sigma_{ZT}, \quad (7)$$

corresponding to PyQUDA gamma indices [15, 8, 7, 1, 14, 2, 13, 4, 11, 0, 9, 3, 5, 10, 6, 12].

3.3 Einsum Contraction Pattern

The array layout of a PyQUDA propagator is

$$\text{prop.data}[w, t, z, y, x_{\text{cb}}, \text{spin}_{\text{sink}}, \text{spin}_{\text{src}}, \text{color}_{\text{sink}}, \text{color}_{\text{src}}], \quad (8)$$

where w is the even-odd parity index (checkerboard), t, z, y, x_{cb} are space-time coordinates, and the remaining four indices carry spin and color.

The antiquark line is constructed as (**_meson_backward_line**):

$$\text{bw_prop}[w, t, z, y, x_{\text{cb}}, j, i, b, a] = (\gamma_5)_{ki} \text{prop.data}[w, t, z, y, x_{\text{cb}}, l, a, b]^* (\gamma_5)_{lj}. \quad (9)$$

The contraction proceeds in two einsum steps:

1. Attach sink gamma to backward line:

$$\text{bw_prop}_\Gamma[g, w, t, z, y, x_{\text{cb}}, j, m, c, f] = \text{bw_prop}[w, t, z, y, x_{\text{cb}}, j, i, c, f] (\Gamma_g)_{im}. \quad (10)$$

2. Full contraction with source gamma and color/spin trace:

$$\text{corr_local}[g, w, t, z, y, x_{\text{cb}}] = \text{bw_prop}_\Gamma[g, w, t, z, y, x_{\text{cb}}, j, i, a, b] \text{prop.f.data}[w, t, z, y, x_{\text{cb}}, l, b, a] (\Gamma_{\text{src}})_{lj}. \quad (11)$$

The momentum phase is applied via a Fourier-phase tensor **phases** $[q, w, t, z, y, x_{\text{cb}}]$ generated by **MomentumPhase.getPhases** with momentum arguments p_{xyz} and source position x_0 . The phase convention in the code uses $e^{-i\mathbf{p}\cdot(\mathbf{x}-\mathbf{x}_0)}$: for the two-point function, negated three-momenta $[-p_x, -p_y, -p_z]$ are passed to the phase generator, so that

$$\text{phases}[q, w, t, z, y, x_{\text{cb}}] \triangleq e^{-i\mathbf{p}_q\cdot(\mathbf{x}-\mathbf{x}_0)}. \quad (12)$$

The final momentum-projected correlator is

$$\text{corr}[g, q, t] = \sum_{w, z, y, x_{\text{cb}}} \text{phases}[q, w, t, z, y, x_{\text{cb}}] \text{corr_local}[g, w, t, z, y, x_{\text{cb}}]. \quad (13)$$

3.4 Boosted Smearing

Two source-smeared, point-sink propagators are defined by

$$Q_+^{SP} = D^{-1} \mathcal{S}_+ \delta_{x_0}, \quad Q_-^{SP} = D^{-1} \mathcal{S}_- \delta_{x_0}. \quad (14)$$

The positive line is the spectator and the negative line is the active operator line. Equal boosts require one inversion and a copy; unequal boosts require two independent inversions. For the two-point function they are boosted-smeared at the sink using their corresponding endpoint kernels:

$$Q_+^{SS} \leftarrow \text{boosted_smearing}(Q_+^{SP}, w, \text{pos_boost}), \quad (15)$$

$$Q_-^{SS} \leftarrow \text{boosted_smearing}(Q_-^{SP}, w, \text{neg_boost}). \quad (16)$$

The implementation in `boosted_smearing_pyquda.py` uses a spatial FFT convolution with the real-space kernel

$$K_{\mathbf{k}}(\mathbf{r}) = \exp\left(-\frac{r_x^2 + r_y^2 + r_z^2}{2w^2}\right) \exp\left[2\pi i \left(\frac{k_x r_x}{L_x} + \frac{k_y r_y}{L_y} + \frac{k_z r_z}{L_z}\right)\right]. \quad (17)$$

Here $\mathbf{k} = (k_x, k_y, k_z)$ is the integer vector supplied by `pos_boost` or `neg_boost`, while $\mathbf{r}$ is the centered periodic global coordinate on the current MPI rank. For each spin-color column of a fermion or propagator the smeared field is

$$\psi_{\mathbf{k}}^{\text{smear}}(\mathbf{x}, t) = \mathcal{F}_{3d}^{-1} [\mathcal{F}_{3d}[\psi](\mathbf{p}, t) \mathcal{F}_{3d}[K_{\mathbf{k}}](\mathbf{p})] (\mathbf{x}, t). \quad (18)$$

The FFT is three-dimensional and spatial only; the same kernel is applied on each Euclidean time slice. The current implementation assumes identity gauge for this smearing step, i.e. no explicit gauge rotation is applied before or after the FFT.

4 Three-Point qTMD Function (Pre-Sequential-Trick)

4.1 Physical Correlator

The connected pion qTMD three-point function, before applying the sequential source trick, reads

$$\begin{aligned} C_3^\Gamma(q, b_\perp, b_z; \tau; p_f, t_{\text{sep}}) = & \sum_{\mathbf{x}} \sum_{\mathbf{y}} e^{-i\mathbf{q} \cdot (\mathbf{x} - \mathbf{x}_0)} e^{-i\mathbf{p}_f \cdot (\mathbf{y} - \mathbf{x}_0)} \\ & \times \text{Tr}_{\text{spin, color}} \left[S_{\text{anti}}(\mathbf{y}, t_{\text{sep}}; \mathbf{x}_0, 0) \Gamma_{\text{sink}} S_q(\mathbf{y}, t_{\text{sep}}; \mathbf{x}, \tau) \Gamma_{\text{ins}} \mathcal{O}_{b_\perp, b_z} S_q(\mathbf{x}, \tau; \mathbf{x}_0, 0) \Gamma_{\text{src}} \right]. \end{aligned} \quad (19)$$

Here $\mathcal{O}_{b_\perp, b_z}$ is the nonlocal displacement operator acting on the forward quark line. Unlike the pion electromagnetic form factor (EMFF), where the insertion is a local current at a single space-time point, the qTMD insertion is nonlocal in space: the quark field at $(\tau, \mathbf{x})$ is shifted by $(b_\perp \mathbf{e}_\perp + b_z \mathbf{e}_z)$. This is the defining difference from the EMFF three-point function.

The momenta satisfy $\mathbf{p}_i$ (initial), $\mathbf{p}_f$ (final sink), and $\mathbf{q}$ (insertion) with $\mathbf{q} = \mathbf{p}_f - \mathbf{p}_i$ in the forward matrix element.

4.2 Why the Sequential Trick is Needed

Computing Eq. (19) directly would require a separate propagator inversion for every $(\mathbf{q}, b_\perp, b_z, \tau)$ combination, which is prohibitively expensive. The fixed-sink sequential source method absorbs the entire sink sum (over $\mathbf{y}$) into a single propagator inversion, after which the insertion-side contraction becomes a simple two-point-like correlation.

5 Sequential Source

5.1 Meson Sequential Source Construction

The meson fixed-sink sequential source is built in `create_meson_bw_seq_pyquda`. Given the sink-smeared propagator Q_+^{SS} (the positive-boost spectator, already positive-boost smeared at the sink), the source position x_0 , the sink momentum p_f , the sink-source separation t_{sep} , and the sink Dirac matrix Γ_{sink} , the right-hand side is placed on the sink time slice $t_{\text{sink}} = t_0 + t_{\text{sep}}$:

$$\eta_{\text{seq}}(\mathbf{y}; p_f, t_{\text{sep}}) = \delta_{t_y, t_{\text{sink}}} \Phi_{p_f}(\mathbf{y} - \mathbf{x}_0) \Gamma_{\text{seq}} S_q(\mathbf{y}, t_{\text{sink}}; \mathbf{x}_0, 0), \quad (20)$$

$$\Gamma_{\text{seq}} = \gamma_5 \Gamma_{\text{sink}}^\dagger \gamma_5. \quad (21)$$

The phase function $\Phi_{p_f}(\mathbf{r}) = e^{-i\mathbf{p}_f \cdot \mathbf{r}}$ is generated by `MomentumPhase.getPhases` with the same sign convention as the two-point function. The time slice restriction is enforced by `pyquda_utils.source.sequential12`, which masks all time slices except t_{sink} .

For the smeared-smeared three-point function, the source above is smeared once more before the sequential inversion:

$$\eta_{\text{seq}}^{SS} = \mathcal{S}_{w, \mathbf{k}_{\text{neg}}} [\delta_{t_y, t_{\text{sink}}} \Phi_{p_f}(\mathbf{y} - \mathbf{x}_0) \Gamma_{\text{seq}} Q_+^{SS}(\mathbf{y}, t_{\text{sink}}; \mathbf{x}_0, 0)]. \quad (22)$$

The input line Q_+^{SS} carries the positive spectator endpoint smearing. The outer $\mathcal{S}_{w, \mathbf{k}_{\text{neg}}}$ supplies the negative active endpoint smearing absorbed into the sequential propagator. At the operator insertion the bilocal displacement or Wilson transporter acts on Q_-^{SP} ; the insertion itself is not Gaussian smeared. This is the same line convention used by connected pion EMT.

Only the positive-spectator/negative-active orientation is produced. The opposite orientation requires exchanging the two complete lines, including both source inversions and both endpoint smearings. Swapping only one boost does not define that second contribution.

The sequential propagator solves

$$D S_{\text{seq}}(x; x_0) = \eta_{\text{seq}}(x), \quad (23)$$

where D is the Wilson-clover Dirac operator. After inversion, γ_5 -hermiticity yields the backward sequential line used in contraction:

$$S_{\text{seq}}^{\text{anti}}(x, x_0; p_f, t_{\text{sep}}) = \gamma_5 S_{\text{seq}}(x, x_0)^\dagger \gamma_5. \quad (24)$$

5.2 Sink-Summed Three-Point Contraction

With the sequential propagator, the sink sum is absorbed and the three-point function reduces to a two-point-like form:

$$C_3^\Gamma(q, b_\perp, b_z; \tau; p_f, t_{\text{sep}}) = \sum_{\mathbf{x}} e^{-i\mathbf{q} \cdot (\mathbf{x} - \mathbf{x}_0)} \text{Tr}_{\text{spin, color}} \left[S_{\text{seq}}^{\text{anti}}(\mathbf{x}, \tau; \mathbf{x}_0, 0) \Gamma_{\text{ins}} \mathcal{O}_{b_\perp, b_z} S_q(\mathbf{x}, \tau; \mathbf{x}_0, 0) \Gamma_{\text{src}} \right]. \quad (25)$$

The insertion gamma Γ_{ins} scans all 16 Dirac structures (the same set as in Sec. 3), while Γ_{src} is the explicit canonical label 5 by default and Γ_{sink} is set by the application (e.g., γ_5 for the pseudoscalar pion).

The shared `contract_pion_gamma_scan_from_backward_line` kernel mirrors the two-point function:

$$\text{sink_inserted}[g, w, t, z, y, x_{\text{cb}}, j, m, c, f] = S_{\text{seq}}^{\text{anti}}[w, t, z, y, x_{\text{cb}}, j, i, c, f] (\Gamma_g)_{im}, \quad (26)$$

$$\text{corr_local}[g, w, t, z, y, x_{\text{cb}}] = \text{sink_inserted}[g, w, t, z, y, x_{\text{cb}}, j, i, a, b] (\text{shifted_prop})[w, t, z, y, x_{\text{cb}}] \quad (27)$$

Here `shifted_prop` is the negative active propagator Q_-^{SP} with the nonlocal displacement operator already applied (see Sec. 6).

6 CG qTMD Operator

6.1 Coordinate-Gauge Displacement

For the CG (coordinate-gauge) qTMD path, the nonlocal operator is approximated by a simple lattice shift, with *no explicit gauge link* inserted:

$$\mathcal{O}_{b_\perp, b_z} S_q(x, x_0) = S_q(x + b_\perp \mathbf{e}_\perp + b_z \mathbf{e}_z, x_0). \quad (28)$$

This corresponds to working in a gauge where the spatial links are (at least approximately) set to unity, so the Wilson line is trivial. The physical interpretation is that the gauge has been fixed to a smooth gauge (e.g., Coulomb or Landau gauge) before the correlation function is computed.

6.2 Transverse Direction Scan

The transverse displacement direction $\mathbf{e}_\perp$ is scanned over the two transverse lattice axes:

- Direction 0: $\mathbf{e}_\perp = \mathbf{e}_x$ (stored as `b_X` in HDF5).
- Direction 1: $\mathbf{e}_\perp = \mathbf{e}_y$ (stored as `b_Y` in HDF5).

The code contracts both directions separately with independent propagator copies so that successive shifts remain small and error accumulation is minimized.

6.3 Wilson Line Index Convention

Each displacement is labeled by a four-component Wilson line index:

$$\mathbf{W_index} = [b_T, b_z, \eta, \text{direction}], \quad (29)$$

where:

- b_T : transverse displacement in lattice units (0 to `b_T`).
- b_z : longitudinal displacement in lattice units (positive and negative, up to $\pm \mathbf{b}_z$).
- η : staple length parameter (set to 0 for straight-line paths in current implementation).
- direction: 0 for $\mathbf{e}_x$, 1 for $\mathbf{e}_y$.

The shared incremental shift operation `qtmd_operator_utils.shift_qtmd_cg` computes

$$\Delta b_T = \text{round}(b_T^{\text{current}} - b_T^{\text{previous}}), \quad \Delta b_z = \text{round}(b_z^{\text{current}} - b_z^{\text{previous}}), \quad (30)$$

and applies `prop.f.shift( $\Delta b_T$ , direction).shift( $\Delta b_z$ , 2)`, where direction 2 is the z -axis. The round function handles cases where b_T or b_z could be non-integer (though in current use they are integers).

7 PDF Operators ($\mathbf{b}_T = 0$)

The PDF path is the straight- z special case with $b_\perp = 0$:

$$\mathbf{b} = b_z \mathbf{e}_z. \quad (31)$$

Two variants are supported, selected by the boolean parameter `gauge_invariant`:

7.1 CG PDF (Coordinate Gauge)

The CG PDF operator is identical in spirit to the CG qTMD operator (Sec. 6) but restricted to the z direction:

$$\mathcal{O}_{b_z}^{\text{CG}} S_q(x, x_0) = S_q(x + b_z \mathbf{e}_z, x_0). \quad (32)$$

No gauge links are included. This is implemented by the same `shift_qtmd_cg` function, which applies a lattice shift in the z direction (direction index 2).

7.2 GI PDF (Gauge Invariant)

The GI PDF operator inserts explicit gauge-covariant shifts that build up the Wilson line incrementally. For a unit step in the $+z$ direction:

$$\psi'(x) = U_z(x) \psi(x + \hat{z}), \quad (33)$$

and for a unit step in the $-z$ direction:

$$\psi'(x) = U_{-z}(x) \psi(x - \hat{z}) = U_z^\dagger(x - \hat{z}) \psi(x - \hat{z}). \quad (34)$$

The implementation in `shift_propagator_pdf_gi` applies the gauge-covariant derivative `gauge.pure_gauge.covDev(fermion, dir)` spin-color by spin-color:

- `covDev(fermion, 2)`: covariant shift in $+z$ ($\Delta b_z = +1$).
- `covDev(fermion, 6)`: covariant shift in $-z$ ($\Delta b_z = -1$).

The PyQUDA direction convention is: 0, 1, 2, 3 for $+x, +y, +z, +t$ and 4, 5, 6, 7 for $-x, -y, -z, -t$.

An important implementation detail is that only nearest-neighbor ($\Delta b_z = \pm 1$) incremental changes are supported. If the ordered Wilson line list contains a jump larger than one lattice unit, the code raises a `ValueError` rather than silently producing an incorrect path. This constraint is satisfied by construction of the Wilson line index list (see Sec. 8).

At zero separation the local limit is common to the qTMD, CG-PDF, GI-PDF, and EMFF contractions:

$$b_T = b_z = \eta = 0, \quad \mathbf{q} = 0 \quad \implies \quad C_3^{\text{qTMD,CG}} = C_3^{\text{PDF,CG}} = C_3^{\text{PDF,GI}} = C_3^{\text{EMFF}} \quad \text{for the same local current } \Gamma \quad (35)$$

This is a useful diagnostic of the phase, gamma, and sequential-source conventions. In the S8T32 smoke test the $\Gamma = \gamma_4$ local-current datasets agree with the pion EMFF output at machine precision.

8 Wilson Line Index Lists

8.1 TMD Wilson Line Indices

The shared function `create_cg_qtmd_wilsonline_index_lists` returns two lists: `index_list_trans0` (transverse direction 0, i.e., x) and `index_list_trans1` (transverse direction 1, i.e., y).

For a given maximum displacement $(b_T^{\max}, b_z^{\max})$, the index pairs are enumerated as:

```
for bz in 0 .. b_z:
    for bT in 0 .. b_T:
        add [bT, bz, 0, 0]    and    [bT, bz, 0, 1]
        if bz != 0:
            add [bT, -bz, 0, 0]    and    [bT, -bz, 0, 1]
```

This generates all (b_T, b_z) pairs in the first quadrant and mirror- z half of the (b_T, b_z) plane. Both positive and negative b_z are included to capture the full Fourier range for the z -direction momentum projection.

8.2 Reordering for Minimal Adjacent Differences

After enumeration, an internal helper in `qtmd_operator_utils.py` reorders each list to minimize the maximum absolute difference in (b_T, b_z) between adjacent indices. The algorithm:

1. Sort by (b_T, b_z) lexicographically.
2. Walk through the sorted list; if the jump from index i to $i + 1$ exceeds one lattice unit in either coordinate, search ahead for a closer candidate and swap it into position $i + 1$.

This greedy reordering reduces the cumulative lattice displacement traversed during incremental propagator shifts, which minimizes numerical noise accumulation and improves code efficiency since smaller shifts are cheaper in the PyQUDA propagator implementation.

8.3 PDF Wilson Line Indices

The shared function `create_pdf_wilsonline_index_list` returns a single list (because $b_T = 0$ always):

```
for bz in 0 .. b_z:
    add [0, bz, 0, 0]
for bz in 0 .. b_z:    # skip bz=0 (already added above)
    if bz != 0:
        add [0, -bz, 0, 0]
```

The list starts at $b_z = 0$ (the unshifted propagator) and incrementally walks out to $\pm b_z^{\max}$ in unit steps. Since b_T is identically zero and the b_z changes are by construction ± 1 or 0 (resetting to $b_z = 0$ between the positive and negative branches), no reordering is needed.

For the GI PDF path, the reset to $b_z = 0$ between branches is essential: it restarts the propagator from the original (unshifted) copy and builds the negative- b_z branch from scratch, avoiding cumulative gauge-link noise from traversing the entire length $+b_z^{\max}$ to $-b_z^{\max}$.

9 Code Implementation Details

The core implementation resides in `pyquda_measurement_utils/pion_qTMD_vibe_develop.py`, in the class `pion_TMD`. Below we document each major method.

9.1 `__init__(parameters)`

Constructs the lightweight contraction object from a parameter dictionary with keys:

- `p_2pt`: momentum list for the two-point function.
- `width`: Gaussian smearing width.
- `pos_boost`, `neg_boost`: boosted smearing momentum boosts for quark and antiquark lines.

The application layer owns η , b_z , b_T , insertion momenta, t_{sep} , and output scheduling. It constructs the Wilson-index lists with the neutral `qtm_d_operator_utils` functions and passes them to the contraction object.

9.2 `contract_2pt_pion`

```
def contract_2pt_pion(self, latt_info, spectator_prop, active_prop, phases, tag
,
                      src_gamma="5"):
```

Workflow:

1. Sink-smear `spectator_prop` with `pos_boost` and `active_prop` with `neg_boost`.
2. Call the shared pion C2 kernel, which constructs the γ_5 -hermitian backward line, scans the 16 sink Gamma channels one at a time, Fourier-projects, and gathers across MPI ranks.

3. Save to HDF5 via `save_proton_c2pt_hdf5`.

The output shape is `[gamma, momentum, time]`.

9.3 `contract_qTMD_CG`

```
def contract_qTMD_CG(self, latt_info, active_prop, seq_bw_prop, phases,
                     W_index_list_dir0, W_index_list_dir1,
                     src_gamma="5"):
```

Workflow:

1. Prepare gamma stacks (Γ_g and Γ_{src}) and phases.
2. Construct sequential backward line $\text{seq_bw_line} = \gamma_5 \text{seq_bw_prop}^\dagger \gamma_5$.
3. For direction 0 (x): iterate over W_index_list_dir0 , incrementally shifting the negative active propagator using `shift_qtmd_cg` and contracting at each step via the shared Gamma-scan kernel.
4. Reset and repeat for direction 1 (y) with W_index_list_dir1 .
5. Stack all results into a numpy array.

Each Wilson index produces an output of shape `[gamma, momentum, time]`; stacking across all N_W Wilson indices yields shape `[N.W, gamma, momentum, time]`.

The per-shift kernel deliberately does not construct a `[16,propagator]` object. It retains only one propagator-sized Gamma-inserted line at a time and accumulates the small `[gamma,momentum,time]` result.

9.4 contract_PDF

```
def contract_PDF(self, latt_info, gauge, active_prop, seq_bw_prop, phases,
                 W_index_list, src_gamma="5",
                 gauge_invariant=True):
```

Same structure as `contract_qTMD_CG` but:

- Uses a single `W_index_list` (no transverse direction separation).
- Resets the negative active propagator to the original copy whenever b_z returns to 0 (handling the split between positive and negative branches).
- Calls `shift_propagator_pdf_gi` when `gauge_invariant=True` or `shift_qtmd_cg` when `gauge_invariant=False`.

9.5 Shared per-shift Gamma scan

```
contract_pion_gamma_scan_from_backward_line(
    latt_info, shifted_prop, seq_bw_line, phases, [src_gamma])
```

The core single-Wilson-index contraction is shared with pion C2, EMFF, qTMDWF, and qDA:

1. `sink.inserted`: `einsum "wtzyxjicf,gim->gwtzyxjmcf"` (attach Γ_g to sequential backward line).
2. `corr_local`: `einsum "gwtzyxjiab,wtzyxilba,glj->gwtzyx"` (contract Γ_g line, shifted forward propagator, Γ_{src}).
3. Momentum projection: `einsum "qwtzyx,gwtzyx->gqt"`.
4. MPI gather: `core.gatherLattice`.

The index conventions are:

- g : gamma index (0–15).
- q : momentum index.
- w : checkerboard parity.
- t, z, y, x_{cb} : space-time coordinates.
- i, j, k, l, m : spin indices.
- a, b, c, f : color indices.

9.6 shift_qtmd_cg

```
def shift_qtmd_cg(field, W_index, W_index_previous):
    dbT = current_b_T - previous_b_T
    dbz = current_bz - previous_bz
    return field.shift(round(dbT), trans_dir).shift(round(dbz), 2)
```

Applies incremental lattice shifts to propagate the forward propagator from one Wilson index to the next. The z -direction is always index 2 (the third spatial axis in PyQUDA's convention).

9.7 shift_propagator_pdf_gi

```
def shift_propagator_pdf_gi(gauge, prop_f, W_index, W_index_previous):
    for spin in range(4):
        for color in range(3):
            fermion = prop_f.getFermion(spin, color)
            if current_bz - previous_bz == 1:
                fermion = gauge.pure_gauge.covDev(fermion, 2) # +z
            elif current_bz - previous_bz == -1:
                fermion = gauge.pure_gauge.covDev(fermion, 6) # -z
            prop_f.setFermion(fermion, spin, color)
```

Applies a gauge-covariant unit shift spin-color by spin-color. The covariant derivative `covDev` parallel-transport the fermion from the neighboring lattice site back to the current site using the gauge link, implementing Eq. (7) of Sec. 7.

9.8 _meson_backward_line

```
def _meson_backward_line(prop):
    return xp.einsum("ij,wtzyxlab,kl->wtzyxkjb",
                    gamma5, prop.data.conj(), gamma5)
```

Implements the γ_5 -hermiticity transformation:

$$(S^{\text{anti}})^{kj}_{ba}(\mathbf{x}) = (\gamma_5)_{ik} (S^{li}_{ab}(\mathbf{x}))^* (\gamma_5)_{lj}. \quad (36)$$

Note that both color and spin indices are transposed by the conjugation, and γ_5 acts on both the source and sink spin spaces.

10 Output Shape Convention and HDF5 Layout

10.1 Raw Contraction Shape

The contraction routines naturally produce arrays of shape

$$[\text{Wilson_index}, \text{gamma}, \text{momentum}, \text{time}]. \quad (37)$$

10.2 Explicit Transpose for HDF5 Write

The Perlmutter application explicitly transposes the contraction array before calling the HDF5 writer:

$$\text{pion_TMDs} \leftarrow \text{np.transpose}(\text{pion_TMDs}, (0, 2, 1, 3)), \quad (38)$$

yielding shape `[Wilson_index, momentum, gamma, time]`. This transpose is the canonical dense HDF5 order expected by `save_connected_qtmd_hdf5`. The writer checks all four axis lengths against the Wilson-index, momentum, complete 16-Gamma, and time metadata before publishing the file.

10.3 Source Time Rolling

The two-point function data is time-rolled by the source time t_0 in `save_proton_c2pt_hdf5`:

$$C_2(t) \leftarrow C_2(t + t_0 \bmod L_t), \quad (39)$$

so that $t = 0$ in the saved data corresponds to the source time slice.

For the three-point function, the application rolls before calling the writer:

$$\text{pion_TMDs} \leftarrow \text{np.roll}(\text{pion_TMDs}, -\text{pos}[3], \text{axis}=-1), \quad (40)$$

and additionally truncates to $\tau \in [0, t_{\text{sep}}]$:

$$\text{pion_TMDs} \leftarrow \text{pion_TMDs}[:, :, :, :t_{\text{sep}}+2]. \quad (41)$$

10.4 HDF5 Directory Layout

The smearing tag identifies only gauge preprocessing and smearing parameters. The pion channel is encoded separately and unambiguously. The two-point file name contains `.src<SRC>`; its attributes store `src_interpolator=<SRC>` and `sink_interpolator=all_16_gamma_scan`. Connected qTMD and PDF three-point file names contain `.src<SRC>.sink<SINK>`. Each operator file contains the complete 16-Gamma axis, and its root attributes store the same source and sink labels together with `operator_gamma_basis=all_16`. The source/sink channel identity also enters the sample-log name, so two channels cannot share resume state even when a custom smearing tag is reused.

10.5 Lightweight Source-Level Resume

At startup rank 0 reads the nonempty sample-log lines once and broadcasts the exact set of completed source tags. A matching source is skipped before any source inversion. The tag is durably appended only after the C2 file and all enabled CG/GI qTMD/PDF files have been closed. The text log is the sole resume state; the workflow deliberately does not test whether HDF5 files remain in the production directory.

This lightweight identity assumes that the enabled products, momentum range, b_T , b_z , η , and the remaining operator grid are unchanged. Changing any such choice requires a new data/setup identity. Concurrent jobs must not append to the same pion qTMD sample log.

The dense HDF5 layout for each three-point operator is:

```
corr                [Wilson, momentum, gamma, time]
gamma_list          [16]
momentum_list       [N_momentum, 4]
wilson_index_list   [N_Wilson, 4]
```

The columns of `wilson_index_list` are $[b_T, b_z, \eta, d_\perp]$, with $d_\perp = 0, 1$ denoting the X, Y directions. PDF files use the same schema with $b_T = 0$. When all products are enabled, one source produces one C2 file and four dense operator files instead of one file per Gamma channel.

11 Boost Smearing Parameters

The boosted smearing parameters `pos_boost` and `neg_boost` are three-component vectors $\mathbf{k}_{\text{boost}} = (k_x, k_y, k_z)$ in units of $2\pi/L$. They control the momentum shift injected into the smearing kernel for the quark (forward) and antiquark (backward) lines respectively.

In the Gaussian smearing kernel

$$K_{\mathbf{k}}(\mathbf{r}) = \exp\left(-\frac{r_x^2 + r_y^2 + r_z^2}{2w^2}\right) \exp\left[2\pi i \left(\frac{k_x r_x}{L_x} + \frac{k_y r_y}{L_y} + \frac{k_z r_z}{L_z}\right)\right], \quad (42)$$

the boost momentum shifts the peak of the smearing kernel in momentum space, effectively selecting quark fields that carry momentum $\mathbf{k}_{\text{boost}}$. This improves the overlap of the interpolating operator with a moving pion state and is essential for achieving good signal at finite boost p_f .

The application accepts both boosts explicitly through `--pos-boost X.Y.Z` and `--neg-boost X.Y.Z`. The zero-boost default reuses one source inversion. For unequal boosts the two propagators in Eq. (14) are inverted separately. The default setup tag encodes both boosts, and HDF5 records `operator_insertion_line=neg_boost` and `boost_line_convention=pos_spectator_neg_active`. Results from the old unequal-boost single-source implementation are not valid under this convention and must be regenerated.

12 Comparison with Proton TMD

The pion qTMD code (`pion_qTMD_vibe_develop.py`) has the same structural organization as the proton qTMD code (`proton_qTMD_pyquda.py`). The key differences arise from the meson- vs.- baryon nature of the hadron:

12.1 Flavor Dimension

- **Proton:** Three quark lines (u, u, d), requiring separate sequential sources for up-quark and down-quark insertions. The flavor index is an explicit dimension of the output data.
- **Pion:** Two quark lines (quark and antiquark) treated symmetrically. There is no flavor dimension; a single sequential source suffices.

12.2 Spin Polarization

- **Proton:** Spin-1/2 particle with polarization dependence. The sequential source construction includes spin projection matrices $P_p S_{zp}, P_p S_{zm}, P_p S_{xp}, P_p S_{xm}, P_p^{\text{unpol}}$ via `PolProjections`.
- **Pion:** Spin-0 particle. No spin polarization is needed. The sequential source is a simple meson backward line (Eq. 20) without spin projection.

12.3 Contraction Structure

- **Proton:** Uses baryon epsilon-tensor contractions ($\epsilon_{abc}\epsilon_{def}$) to form the color-singlet proton state. The two-point function involves two contraction terms (direct and exchange), and the three-point function involves separate diquark contractions (`down_quark_insertion_pyquda`, `up_quark_insertion_pyquda`).
- **Pion:** Uses meson contraction (simple color trace) via `_meson_backward_line` for the antiquark line. The pion is a color singlet $\delta_{ab}\bar{\psi}_a\psi_b$, so the contraction is a single term with no exchange contributions.

12.4 Sequential Source

- **Proton:** `create_bw_seq_pyquda` builds the baryon sequential source, which involves diquark contractions, time slicing, momentum phase insertion, γ_5 -conjugation, boosted smearing, inversion, and final spin projection.
- **Pion:** `create_meson_bw_seq_pyquda` builds the meson sequential source, which is considerably simpler: time-slice the propagator, apply $\Gamma_{\text{seq}} = \gamma_5 \Gamma_{\text{sink}}^\dagger \gamma_5$ to the source spin index, multiply by the momentum phase, invert, and return the propagator.

12.5 Summary Table

Feature	Pion qTMD	Proton qTMD
Hadron type	Meson (spin-0)	Baryon (spin-1/2)
Quark lines	2 ($q + \bar{q}$)	3 (u, u, d)
Flavor dimension	None	2 (up, down)
Spin polarization	None	5 projections
Contraction	<code>_meson_backward_line</code>	ϵ baryon contr.
Sequential source	<code>create_meson_bw_seq</code>	<code>create_bw_seq</code>
Wilson line ops	CG qTMD, CG/PDF, GI/PDF	CG qTMD, CG/PDF, GI/PDF
HDF5 layout	Same as proton	Standard

13 Summary of Key Formulae

For quick reference, the central contracted expressions are:

Two-point (Eq. 5):

$$C_2^g(p, t) = \sum_x e^{-ip \cdot (x - x_0)} \text{Tr}_{\text{sp, col}} [(\gamma_5 S_q^\dagger \gamma_5)(g) \Gamma_g S_q(x, x_0) \Gamma_{\text{src}}].$$

Sink-summed three-point (Eq. 25):

$$C_3^g(q, b, \tau; p_f, t_{\text{sep}}) = \sum_x e^{-iq \cdot (x - x_0)} \text{Tr}_{\text{sp, col}} [S_{\text{seq}}^{\text{anti}}(x, \tau) \Gamma_g \mathcal{O}_b S_q(x, \tau) \Gamma_{\text{src}}].$$

Meson sequential source (Eqs. 20, 21):

$$\eta_{\text{seq}}(y) = \delta_{t_y, t_{\text{sink}}} \Phi_{p_f}(y - x_0) (\gamma_5 \Gamma_{\text{sink}}^\dagger \gamma_5) S_q(y, x_0).$$

CG displacement (Eq. 28):

$$\mathcal{O}_{b_\perp, b_z} S_q(x, x_0) = S_q(x + b_\perp e_\perp + b_z e_z, x_0).$$

GI covariant shift (Sec. 7, unit $+z$):

$$\psi'(x) = U_z(x) \psi(x + \hat{z}), \quad (\text{covDev}(\text{fermion}, 2)).$$

A Gamma Matrix Reference

The dense qTMD/PDF dataset `corr[wilson, momentum, gamma, time]` stores the raw PyQUDA bit-mask Gamma basis actually used in the contraction. It is identified by the HDF5 attribute

```
gamma_basis_schema = pyquda.bitmask16_with_physics_transform.v1.
```

This identifier is a version key; the complete convention is carried by the same file in `gamma_list`, `gamma_pyquda_ids`, `gamma_matrices`, and `physical_from_pyquda`. In particular, the table below describes the *raw matrix used in production*, not an already converted physics label.

Index	Label	Raw production matrix
0	5	γ_5
1	T	γ_T (temporal)
2	T5	$-\gamma_T\gamma_5$
3	X	γ_X
4	X5	$\gamma_X\gamma_5$
5	Y	γ_Y
6	Y5	$-\gamma_Y\gamma_5$
7	Z	γ_Z
8	Z5	$\gamma_Z\gamma_5$
9	I	Identity
10	SXT	$\frac{1}{2}[\gamma_X, \gamma_T] = \gamma_X\gamma_T$
11	SXY	$\frac{1}{2}[\gamma_X, \gamma_Y] = \gamma_X\gamma_Y$
12	SXZ	$\frac{1}{2}[\gamma_X, \gamma_Z] = \gamma_X\gamma_Z$
13	SYT	$\frac{1}{2}[\gamma_Y, \gamma_T] = \gamma_Y\gamma_T$
14	SYZ	$\frac{1}{2}[\gamma_Y, \gamma_Z] = \gamma_Y\gamma_Z$
15	SZT	$\frac{1}{2}[\gamma_Z, \gamma_T] = \gamma_Z\gamma_T$

The physics-labelled basis is defined file-by-file by

$$\Gamma_A^{\text{phys}} = \sum_B (\text{physical_from_pyquda})_{AB} \Gamma_B^{\text{raw}}. \quad (43)$$

For schema v1 this transform is the identity except for

$$\Gamma_{Y5}^{\text{phys}} = -\Gamma_{Y5}^{\text{raw}} = \gamma_Y\gamma_5, \quad \Gamma_{T5}^{\text{phys}} = -\Gamma_{T5}^{\text{raw}} = \gamma_T\gamma_5. \quad (44)$$

Thus an analysis must apply this transform before treating every axial label uniformly as $\gamma_\mu\gamma_5$. The raw tensor channels contain no factor of i ; the Hermitian convention $i[\gamma_\mu, \gamma_\nu]/2$ is obtained by multiplying the corresponding stored channel by i . The repository-wide schema registry and an HDF5 conversion example are given in `docs/EMT_gamma_and_raw_bilinears.md`.

B Wilson Line Index Convention

Connected GI qTMD Update

The connected pion qTMD code now supports both the older Coulomb-gauge coordinate-shift qTMD and a gauge-invariant staple qTMD. The canonical `application/pion.TMD/perlmutter` workflow can run `CG_qTMD`, `GI_qTMD`, `GI_PDF`, and `CG_PDF` from the same forward and sequential propagators.

For the connected `GI_qTMD` operator, the Wilson index remains

$$W = [b_T, b_z, \eta, d_\perp], \quad (45)$$

where $d_\perp = 0, 1$ denotes the transverse x or y direction. The staple is implemented with the fixed-length convention

$$x \rightarrow x + \left(\eta + \frac{b_z}{2}\right) \hat{z}, \quad (46)$$

$$\rightarrow x + \left(\eta + \frac{b_z}{2}\right) \hat{z} + b_T \hat{e}_{d_\perp}, \quad (47)$$

$$\rightarrow x + b_z \hat{z} + b_T \hat{e}_{d_\perp}. \quad (48)$$

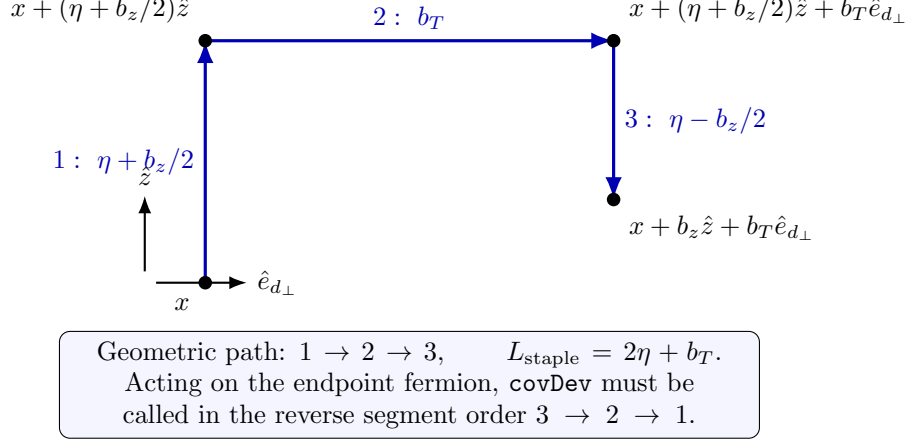


Figure 1: Fixed-length GI qTMD staple geometry, drawn for a representative positive b_z . Segment 3 has signed displacement $(b_z/2 - \eta)\hat{z}$; its displayed downward length is $\eta - b_z/2$. The same formulas cover negative b_z .

Figure 1 distinguishes the geometric path from the operator-composition order. With $D_\mu\psi(x) = U_\mu(x)\psi(x + \hat{\mu})$, the rightmost displacement acts on the endpoint field first, so the implementation traverses the segment list in reverse when constructing the transporter.

Thus the total staple length is

$$L_{\text{staple}} = 2\eta + b_T, \quad (49)$$

independent of b_z . This is useful for renormalization studies because different even b_z values at fixed η and b_T share the same total staple length. The constraints are

$$b_z \in 2\mathbb{Z}, \quad \eta \geq \frac{|b_z|}{2}, \quad b_T \geq 0. \quad (50)$$

The implementation uses the same tested neutral helper convention as the disconnected qTMD one-point code: first build reusable gauge-only staple transporters, then apply each transporter to the endpoint-shifted forward propagator. The shared `qtmd_operator_utils.py` interface contains

- `create_gi_qtmd_wilsonline_index_lists(...)`,
- `build_gi_qtmd_staple_links(...)`,
- `apply_gi_qtmd_staple_to_propagator(...)`.

The HDF5 tag for the connected pion gauge-invariant staple output is `GI-qTMD.ex`. The HDF5 internal path still uses the existing qTMD writer convention

$$b_X/\eta\eta/b_T b_T \quad \text{or} \quad b_Y/\eta\eta/b_T b_T, \quad (51)$$

with the full W list saved by the writer. For the local/PDF sanity limits, the expected equality is

$$C_3^{\text{GI-qTMD}}(b_T = 0, b_z = \pm 2\eta, \eta) = C_3^{\text{GLPDF}}(b_z = \pm 2\eta), \quad (52)$$

up to the common momentum, gamma, source, and sink conventions.

The corrected composition order was validated on the repository `S8T8_wilson_b6.0` gauge after one production-style four-dimensional HYP step. One-rank (1.1.1.1) and four-rank (2.2.1.1) runs gave bitwise-identical gathered staple fields. Across positive and negative b_z , both transverse directions, and the straight-PDF limits, the largest cached-versus-direct relative L_2 error was 3.62×10^{-16} . Field covariance, cached-link endpoint covariance, and contracted-loop gauge invariance were all below 4.08×10^{-16} in relative L_2 error, while the link-free coordinate-shift control changed by 0.29–0.96. The former segment order differs from the

corrected result by 0.438–0.675 on the non-straight actual-gauge paths. A minimal connected GI-qTMD comparison produced the same 16 HDF5 files and 192 datasets in both layouts; its maximum dataset relative L_2 difference was 1.31×10^{-13} , maximum absolute difference was 1.42×10^{-16} , and maximum solver true residual was 8.54×10^{-16} .

Field	Values	Meaning
b_T	$0, 1, \dots, b_T^{\max}$	Transverse displacement
b_z	$-b_z^{\max}, \dots, +b_z^{\max}$	Longitudinal displacement; even for GI qTMD
η	0 for CG, $\eta \geq b_z /2$ for GI qTMD	Staple length parameter
direction	0 or 1	0 for x , 1 for y

References

- [1] X. Ji, *Parton Physics on a Euclidean Lattice*, Phys. Rev. Lett. **110**, 262002 (2013), arXiv:1305.1539 [hep-ph].
- [2] X. Ji, *Parton Physics from Large-Momentum Effective Field Theory*, Sci. China Phys. Mech. Astron. **57**, 1407–1412 (2014), arXiv:1404.6680 [hep-ph].
- [3] X. Ji, Y.-S. Liu, Y. Liu, J.-H. Zhang, and Y. Zhao, *Large-Momentum Effective Theory*, Rev. Mod. Phys. **93**, no. 3, 035005 (2021), arXiv:2004.03543 [hep-ph].
- [4] A. V. Radyushkin, *Quasi-parton distribution functions, momentum distributions, and pseudo-parton distribution functions*, Phys. Rev. D **96**, 034025 (2017), arXiv:1705.01488 [hep-ph].
- [5] K. Orginos, A. Radyushkin, J. Karpie, and S. Zafeiropoulos, *Lattice QCD exploration of parton pseudo-distribution functions*, Phys. Rev. D **96**, 094503 (2017), arXiv:1706.05373 [hep-ph].